

7. System Testing

Verification and Validation (V&V) confirm that the software system matches its SRS specifications. Testing was executed at three levels: unit testing of models, integration testing of API endpoints, and validation testing of client workflows.

7.1 Unit Testing

Unit tests verify the correctness of isolated, small pieces of logic — primarily database constraints, model methods, and helper utilities.

Sample Django Model Unit Test Case

```
from django.test import TestCase
from users.models import User

class UserModelTest(TestCase):
    def setUp(self):
        self.user = User.objects.create_user(
            email="vendor@test.com",
            username="testvendor",
            password="secret123",
            role="vendor"
        )

    def test_role_assignment(self):
        self.assertEqual(self.user.role, "vendor")
        self.assertTrue(self.user.is_active)

    def test_password_hashing(self):
        self.assertTrue(self.user.check_password("secret123"))
        self.assertNotEqual(self.user.password, "secret123") # check hashing
```

Unit Test Execution Results

Test ID	Module Tested	Verification Description	Status
UT-01	User Model	Verify role choice field accepts 'customer', 'vendor', and 'admin'.	PASSED

UT-02	User Model	Assert passwords are encrypted using PBKDF2 algorithm in database.	PASSED
UT-03	User Model	Assert unique constraint is enforced on the user email column.	PASSED
UT-04	User Model	Check string serialization returns email and role details.	PASSED
UT-05	Vendor Model	Verify that a new vendor has <code>is_approved=False</code> by default.	PASSED
UT-06	Product Model	Verify decimal precision values in product price field.	PASSED
UT-07	Product Model	Assert validation rule blocks negative integer stock listing.	PASSED
UT-08	Order Model	Confirm status default is set to 'pending' on instantiation.	PASSED
UT-09	OrderItem Model	Verify <code>get_subtotal()</code> returns correct quantity * price value.	PASSED

7.2 Integration Testing

Integration tests verify the communication channels between subsystems — such as verifying endpoints retrieve correctly from tables and respect user access tokens.

Sample DRF API client integration test

```
from rest_framework.test import APITestCase
from rest_framework import status

class ProductApiTest(APITestCase):
    def test_public_product_listing(self):
        response = self.client.get("/api/products/")
        self.assertEqual(response.status_code, status.HTTP_200_OK)

    def test_customer_blocked_write(self):
        # Customer role is authenticated but RBAC blocks write = 403
        self.client.force_authenticate(user=customer_user)
        response = self.client.post("/api/products/", {"name": "Pen", "price": 10})
        self.assertEqual(response.status_code, status.HTTP_403_FORBIDDEN)
```

Integration Scenarios

Test ID	Integration Scenario	Endpoint & Payload details	Expected HTTP
IT-01	Public Listing	GET request to <code>/api/products/</code> without active tokens.	200 OK
IT-02	Approved Vendor Post	POST request to <code>/api/products/</code> with vendor token and name/price payload.	201 Created
IT-03	Customer Product Block	POST request to <code>/api/products/</code> using token belonging to customer role. Customer is authenticated but <code>IsVendorOrReadOnly</code>	403 Forbidden

		y blocks write access.	
IT-04	Unauthenticated Me Block	GET request to <code>/api/auth/me/</code> without authorization headers.	401 Unauthorized
IT-05	Vendor Product Protection	PUT request to <code>/api/products/{id}/</code> for item owned by a different vendor profile.	403 Forbidden
IT-06	Admin Vendor Approval	PATCH request to <code>/api/vendors/{id}/</code> toggle <code>is_approved</code> by Admin.	200 OK

7.3 Validation Testing (Postman Tests)

Validation testing checks the software against final customer expectations. 16 distinct test suites were ran using Postman to assert API request shapes and validation error handling:

ID	Suite Name	Postman Check Description	Result
VAL-01	Register Validation	Assert status 400 when registering with duplicate email string.	SUCCESS
VAL-02	Login Schema	Check schema response has both "access" and "refresh" keys.	SUCCESS

VAL-03	Bad Login	Check status 401 when sending wrong credentials.	SUCCESS
VAL-04	Token Expiry	Send requests with expired access keys to assert server block.	SUCCESS
VAL-05	Token Rotate	POST to /api/auth/refresh/ with refresh token, check new access token returns.	SUCCESS
VAL-06	Customer Checkout	POST order payload, assert item stock counts decrease.	SUCCESS
VAL-07	Over-Order Block	POST order quantity higher than stock limit, assert 400 bad request.	SUCCESS
VAL-08	Order History	GET /api/orders/ as customer, check only customer's items return.	SUCCESS
VAL-09	Vendor Product CRUD	Create, read, update, and delete products using approved vendor token.	SUCCESS
VAL-10	Unapproved Product Block	Assert vendor account in pending state cannot save product rows.	SUCCESS

VAL-11	Category Manage	Assert only Admins can create Category records.	SUCCESS
VAL-12	Search Keyword	Query <code>/api/products/?search=iphone</code> , verify matching list returns.	SUCCESS
VAL-13	Filter Category	Query <code>/api/products/?category=electronics</code> , check output filters.	SUCCESS
VAL-14	Vendor Dashboard Stats	Query stats endpoint as Vendor, assert only vendor's totals are calculated.	SUCCESS
VAL-15	Admin Dash View	Query stats as Admin, assert system-wide user and revenue sums are shown.	SUCCESS
VAL-16	Dashboard Role Isolation	Query <code>/api/dashboard/stats/</code> as Customer, assert response only contains customer-scoped fields (<code>orders</code> , <code>pending</code> , <code>delivered</code> , <code>spent</code>) not admin or vendor fields.	SUCCESS

7.4 Debugging Log

Below are five notable developer bugs resolved during final system integration.

Bug / Symptom	Root Cause	Resolution Applied
CORS Policy Blocking API Requests	Browser blocked requests from <code>localhost:5173</code> to <code>localhost:8000</code> because of missing headers.	Installed <code>django-cors-headers</code> , added <code>CorsMiddleware</code> first, and set <code>CORS_ALLOWED_ORIGINS</code> .
Protected Endpoints return 401	JWT tokens were stored in local storage but never attached to outgoing Axios calls.	Added a request interceptor in <code>src/services/api.js</code> to inject access token into Authorization header.
RelatedObjectDoesNotExist on Product Save	A vendor profile record (holding shop name and approval status) was not created when a User registered with role='vendor'.	Implemented a Django <code>post_save</code> signal in <code>vendors/models.py</code> to automatically initialize profiles.
IntegrityError: price NOT NULL in OrderItem	Order serializer failed to write price at purchase time into OrderItem rows.	Modified <code>create()</code> in <code>OrderSerializer</code> to extract the product price snapshot and write it to OrderItem rows.
Cart Totals Showing NaN in Browser	Django decimal prices are serialized as strings in JSON. React JS was treating them as strings, causing arithmetic failures.	Wrapped all cart calculations with <code>parseFloat()</code> in <code>CartContext.jsx</code> and product card elements.

7.5 Test Summary Metrics

Test Level	Tests Configured	Verification Count
Unit Testing	Django Model Assertion cases	9 Test Cases

Integration Testing	Django REST Client requests	6 Scenarios
Validation Testing	Postman Client flow checks	16 Scenarios
Total Checks Passed	Stateless verification coverage	31 test flows